

The Basis of Linear Algebra

Workshop – Term 2, Week 8

Programming $\oplus$ Mathematics

Outline

1 Housekeeping

2 Gaussian Elimination

3 Tricks With A^N

4 Diagonalisation

5 XOR Basis

6 Jordan Canonical Form and Matrix Exponential

7 Linear Transformations

8 Appendix and resources

Attendance!

Please scan the attendance form so you can get free food



Problems For Workshop

If you're confident with the content for today, feel free to get started on problems!

There's both a VJudge (for programming) and a worksheet (for maths).

VJudge Link: <https://vjudge.net/contest/826177>

Outline

1 Housekeeping

2 Gaussian Elimination

3 Tricks With A^N

4 Diagonalisation

5 XOR Basis

6 Jordan Canonical Form and Matrix Exponential

7 Linear Transformations

8 Appendix and resources

Gaussian Elimination

Gaussian Elimination

Motivation

As we've learnt in 1141/1131 systems of linear equations can be represented as matrices, meaning they can be solved quickly using Gaussian Elimination (and back substitution).

Gaussian Elimination

Motivation

As we've learnt in 1141/1131 systems of linear equations can be represented as matrices, meaning they can be solved quickly using Gaussian Elimination (and back substitution).

Example

Suppose we wish to solve the equations

$$4x + 2y = 3,$$

$$6y + 7z = 5,$$

$$6x + 9z = 1.$$

Gaussian Elimination

Example

Then we can rewrite the equation as

$$\begin{pmatrix} 4 & 2 & 0 \\ 0 & 6 & 7 \\ 6 & 0 & 9 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \\ 1 \end{pmatrix}.$$

Applying Gaussian Elimination on the augmented matrix, we get

$$\left(\begin{array}{ccc|c} 1 & 0 & 0 & 43/150 \\ 0 & 1 & 0 & 139/150 \\ 0 & 0 & 1 & -2/25 \end{array} \right),$$

which leads us to the answers.

Lights Out

The Game

Imagine a 5×5 grid of lightbulbs. Pressing any bulb toggles its state (on/off) along with its four orthogonal neighbors. Given some initial state, we wish to turn off all the lights.

Lights Out

The Game

Imagine a 5×5 grid of lightbulbs. Pressing any bulb toggles its state (on/off) along with its four orthogonal neighbors. Given some initial state, we wish to turn off all the lights.

- 1 Is this always possible?
- 2 If so, what is our strategy? If not, can you think of a counterexample?
- 3 How can this be generalised for an $N \times M$ grid?

Lights Out

Insight No 1.

Because pressing a button twice is the same as not pressing it at all, each move is a binary decision (0 or 1). Toggling a light corresponds is equivalent to addition modulo 2 (over the field $\mathbb{F}_2$).

Lights Out

Insight No 1.

Because pressing a button twice is the same as not pressing it at all, each move is a binary decision (0 or 1). Toggling a light corresponds to addition modulo 2 (over the field $\mathbb{F}_2$).

Example (3×3 Board)

Let $x_{ij} \in \{0, 1\}$ represent whether we press the cell at row i , column j . For the top-left corner cell $(1, 1)$ to end up off, the sum of its initial state b_{11} and the toggles affecting it must be equivalent (mod 2):

$$x_{11} + x_{12} + x_{21} \equiv b_{11} \pmod{2}.$$

Step 1: Vectorizing the Grid

To use linear algebra, we must "flatten" the $n \times m$ grid into a single column vector $\mathbf{x}$ of length $N = nm$.

The Mapping

We map each cell (i, j) to a unique index k using row-major order:

$$k = (i - 1)m + j$$

A 3×3 board becomes a vector $\mathbf{x} = (x_1, x_2, \dots, x_9)^T$:

x_1	x_2	x_3
x_4	x_5	x_6
x_7	x_8	x_9

 $\Rightarrow$
$$\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_9 \end{pmatrix}$$

Step 2: Building the Equations

Every cell k in the grid has an equation. For a 3×3 grid, the state of the center cell (index 5) is toggled by its neighbors (2, 4, 6, 8) and itself:

- **Cell 1 (Corner):** $x_1 + x_2 + x_4 = b_1 \pmod{2}$
- **Cell 2 (Edge):** $x_1 + x_2 + x_3 + x_5 = b_2 \pmod{2}$
- **Cell 5 (Center):** $x_2 + x_4 + x_5 + x_6 + x_8 = b_5 \pmod{2}$

We've now reduced this problem to solving a system of linear equations.

Step 2: Building the Equations

Every cell k in the grid has an equation. For a 3×3 grid, the state of the center cell (index 5) is toggled by its neighbors (2, 4, 6, 8) and itself:

- **Cell 1 (Corner):** $x_1 + x_2 + x_4 = b_1 \pmod{2}$
- **Cell 2 (Edge):** $x_1 + x_2 + x_3 + x_5 = b_2 \pmod{2}$
- **Cell 5 (Center):** $x_2 + x_4 + x_5 + x_6 + x_8 = b_5 \pmod{2}$

We've now reduced this problem to solving a system of linear equations.

In matrix form $A\mathbf{x} = \mathbf{b}$, the k -th row of A contains a **1** at every index j that toggles cell k .

Lights Out as a Matrix System

The Linear System over $\mathbb{F}_2$

For any $n \times m$ board, we can express the entire puzzle as a system of nm linear equations with nm variables:

$$A\mathbf{x} = \mathbf{b} \pmod{2}$$

- $\mathbf{x}$ is the action vector (which buttons to press).
- $\mathbf{b}$ is the initial board state (1 for ON, 0 for YES).
- A is the **adjacency matrix** of the grid graph, plus the identity matrix (representing that pressing a cell affects itself and its neighbors).

Therefore, finding a solution for a $n \times m$ grid equivalent to solving for $\mathbf{x}$, which can be done by Gaussian Elimination.

Outline

- 1 Housekeeping
- 2 Gaussian Elimination
- 3 Tricks With A^N**
- 4 Diagonalisation
- 5 XOR Basis
- 6 Jordan Canonical Form and Matrix Exponential
- 7 Linear Transformations
- 8 Appendix and resources

Exponentiation

Repeated Multiplication

You're given a positive integer a , as well as a positive integer N . Compute

$$a^N \pmod{10^9 + 7}.$$

Constraints:

- $1 \leq a, N \leq 10^9$.

Exponentiation

Repeated Multiplication

You're given a positive integer a , as well as a positive integer N . Compute

$$a^N \pmod{10^9 + 7}.$$

Constraints:

- $1 \leq a, N \leq 10^9$.

We'd like to just multiply 1 by a N times (reducing mod $10^9 + 7$ as we go).

Exponentiation

Repeated Multiplication

You're given a positive integer a , as well as a positive integer N . Compute

$$a^N \pmod{10^9 + 7}.$$

Constraints:

- $1 \leq a, N \leq 10^9$.

We'd like to just multiply 1 by a N times (reducing mod $10^9 + 7$ as we go).

- What's the time complexity?

Exponentiation

Repeated Multiplication

You're given a positive integer a , as well as a positive integer N . Compute

$$a^N \pmod{10^9 + 7}.$$

Constraints:

- $1 \leq a, N \leq 10^9$.

We'd like to just multiply 1 by a N times (reducing mod $10^9 + 7$ as we go).

- What's the time complexity?

Naive computation is $O(N)$, which is unworkable for $N \sim 10^9$...

Fast Exponentiation

How can we do any less than N multiplications?

Fast Exponentiation

How can we do any less than N multiplications?

- Let's try an example! Try to calculate 3^8 by hand.

Fast Exponentiation

How can we do any less than N multiplications?

- Let's try an example! Try to calculate 3^8 by hand.
- Instead of multiplying by 3 a bunch of times, we can split efficiently –

$$3^8 = 3^4 \cdot 3^4 = (3^2 \cdot 3^2) \cdot (3^2 \cdot 3^2) = 6561.$$

- It's easier to just split our exponent by half and multiply instead :)

Fast Exponentiation

How can we do any less than N multiplications?

- Let's try an example! Try to calculate 3^8 by hand.
- Instead of multiplying by 3 a bunch of times, we can split efficiently –

$$3^8 = 3^4 \cdot 3^4 = (3^2 \cdot 3^2) \cdot (3^2 \cdot 3^2) = 6561.$$

- It's easier to just split our exponent by half and multiply instead :)

This yields a **divide and conquer** idea:

Fast Exponentiation

How can we do any less than N multiplications?

- Let's try an example! Try to calculate 3^8 by hand.
- Instead of multiplying by 3 a bunch of times, we can split efficiently –

$$3^8 = 3^4 \cdot 3^4 = (3^2 \cdot 3^2) \cdot (3^2 \cdot 3^2) = 6561.$$

- It's easier to just split our exponent by half and multiply instead :)

This yields a **divide and conquer** idea:

- Consider the recursive process $f(a, N) := a^N$.

Fast Exponentiation

How can we do any less than N multiplications?

- Let's try an example! Try to calculate 3^8 by hand.
- Instead of multiplying by 3 a bunch of times, we can split efficiently –

$$3^8 = 3^4 \cdot 3^4 = (3^2 \cdot 3^2) \cdot (3^2 \cdot 3^2) = 6561.$$

- It's easier to just split our exponent by half and multiply instead :)

This yields a **divide and conquer** idea:

- Consider the recursive process $f(a, N) := a^N$.
 - If $N = 0$ or 1 , we're done (either 0 or a trivially).

Fast Exponentiation

How can we do any less than N multiplications?

- Let's try an example! Try to calculate 3^8 by hand.
- Instead of multiplying by 3 a bunch of times, we can split efficiently –

$$3^8 = 3^4 \cdot 3^4 = (3^2 \cdot 3^2) \cdot (3^2 \cdot 3^2) = 6561.$$

- It's easier to just split our exponent by half and multiply instead :)

This yields a **divide and conquer** idea:

- Consider the recursive process $f(a, N) := a^N$.
 - If $N = 0$ or 1 , we're done (either 0 or a trivially).
 - Else, if N is even, we should compute $f(a, \lfloor N/2 \rfloor) \cdot f(a, \lfloor N/2 \rfloor)$.

Fast Exponentiation

How can we do any less than N multiplications?

- Let's try an example! Try to calculate 3^8 by hand.
- Instead of multiplying by 3 a bunch of times, we can split efficiently –

$$3^8 = 3^4 \cdot 3^4 = (3^2 \cdot 3^2) \cdot (3^2 \cdot 3^2) = 6561.$$

- It's easier to just split our exponent by half and multiply instead :)

This yields a **divide and conquer** idea:

- Consider the recursive process $f(a, N) := a^N$.
 - If $N = 0$ or 1 , we're done (either 0 or a trivially).
 - Else, if N is even, we should compute $f(a, \lfloor N/2 \rfloor) \cdot f(a, \lfloor N/2 \rfloor)$.
 - Otherwise, compute $f(a, \lfloor N/2 \rfloor) \cdot f(a, \lfloor N/2 \rfloor) \cdot a$.

Fast Exponentiation

How can we do any less than N multiplications?

- Let's try an example! Try to calculate 3^8 by hand.
- Instead of multiplying by 3 a bunch of times, we can split efficiently –

$$3^8 = 3^4 \cdot 3^4 = (3^2 \cdot 3^2) \cdot (3^2 \cdot 3^2) = 6561.$$

- It's easier to just split our exponent by half and multiply instead :)

This yields a **divide and conquer** idea:

- Consider the recursive process $f(a, N) := a^N$.
 - If $N = 0$ or 1 , we're done (either 0 or a trivially).
 - Else, if N is even, we should compute $f(a, \lfloor N/2 \rfloor) \cdot f(a, \lfloor N/2 \rfloor)$.
 - Otherwise, compute $f(a, \lfloor N/2 \rfloor) \cdot f(a, \lfloor N/2 \rfloor) \cdot a$.
- Since we recurse (split) at most $\log N$ times, we can compute $f(a, N)$ in $O(\log N)!$

Fibonacci, Again

Fibonacci (Redux Redux)

Fibonacci, Again

Fibonacci (Redux Redux)

You've earned international acclaim for your ability to find the N 'th Fibonacci number under increasingly tight time-limits.

Fibonacci, Again

Fibonacci (Redux Redux)

You've earned international acclaim for your ability to find the N 'th Fibonacci number under increasingly tight time-limits.

Of course, you remember that F_k is defined recursively, where $F_k = F_{k-1} + F_{k-2}$ for $k \geq 2$, and $F_0 = 0$, $F_1 = 1$.

Fibonacci, Again

Fibonacci (Redux Redux)

You've earned international acclaim for your ability to find the N 'th Fibonacci number under increasingly tight time-limits.

Of course, you remember that F_k is defined recursively, where $F_k = F_{k-1} + F_{k-2}$ for $k \geq 2$, and $F_0 = 0$, $F_1 = 1$.

You recall the days where you relied on $O(\varphi^N)$ recursion, and reflect on the first time you optimised to $O(N)$ with elementary DP. Now, you're beyond such trivialities, and would like to pursue greater challenges better worth your time.

Fibonacci, Again

Fibonacci (Redux Redux)

You've earned international acclaim for your ability to find the N 'th Fibonacci number under increasingly tight time-limits.

Of course, you remember that F_k is defined recursively, where $F_k = F_{k-1} + F_{k-2}$ for $k \geq 2$, and $F_0 = 0$, $F_1 = 1$.

You recall the days where you relied on $O(\varphi^N)$ recursion, and reflect on the first time you optimised to $O(N)$ with elementary DP. Now, you're beyond such trivialities, and would like to pursue greater challenges better worth your time.

Find the N 'th Fibonacci number (modulo $10^9 + 7$).

Fibonacci, Again

Fibonacci (Redux Redux)

You've earned international acclaim for your ability to find the N 'th Fibonacci number under increasingly tight time-limits.

Of course, you remember that F_k is defined recursively, where $F_k = F_{k-1} + F_{k-2}$ for $k \geq 2$, and $F_0 = 0$, $F_1 = 1$.

You recall the days where you relied on $O(\varphi^N)$ recursion, and reflect on the first time you optimised to $O(N)$ with elementary DP. Now, you're beyond such trivialities, and would like to pursue greater challenges better worth your time.

Find the N 'th Fibonacci number (modulo $10^9 + 7$).

Constraints:

- $1 \leq N \leq 10^{18}$.

Fibonacci – Initial Thoughts

Since $N \sim 10^{18}$, we're looking for an $O(1)$ or $O(\log N)$ solution... how???

Fibonacci – Initial Thoughts

Since $N \sim 10^{18}$, we're looking for an $O(1)$ or $O(\log N)$ solution... how???

Recall that there *is* a closed form solution to the linear recurrence:

$$F_N = \frac{\varphi^N - \bar{\varphi}^N}{\sqrt{5}}, \text{ where } \varphi = \frac{1 + \sqrt{5}}{2} \text{ and } \bar{\varphi} = \frac{1 - \sqrt{5}}{2}.$$

Fibonacci – Initial Thoughts

Since $N \sim 10^{18}$, we're looking for an $O(1)$ or $O(\log N)$ solution... how???

Recall that there *is* a closed form solution to the linear recurrence:

$$F_N = \frac{\varphi^N - \bar{\varphi}^N}{\sqrt{5}}, \text{ where } \varphi = \frac{1 + \sqrt{5}}{2} \text{ and } \bar{\varphi} = \frac{1 - \sqrt{5}}{2}.$$

Can we compute φ^N and $\bar{\varphi}^N$ in $O(\log N)$ with fast exp, yielding an $O(\log N)$ solution...?

Fibonacci – Initial Thoughts

Since $N \sim 10^{18}$, we're looking for an $O(1)$ or $O(\log N)$ solution... how???

Recall that there *is* a closed form solution to the linear recurrence:

$$F_N = \frac{\varphi^N - \bar{\varphi}^N}{\sqrt{5}}, \text{ where } \varphi = \frac{1 + \sqrt{5}}{2} \text{ and } \bar{\varphi} = \frac{1 - \sqrt{5}}{2}.$$

Can we compute φ^N and $\bar{\varphi}^N$ in $O(\log N)$ with fast exp, yielding an $O(\log N)$ solution...?

- φ is irrational, and processing non-integers programmatically isn't ideal.

Fibonacci – Initial Thoughts

Since $N \sim 10^{18}$, we're looking for an $O(1)$ or $O(\log N)$ solution... how???

Recall that there *is* a closed form solution to the linear recurrence:

$$F_N = \frac{\varphi^N - \bar{\varphi}^N}{\sqrt{5}}, \text{ where } \varphi = \frac{1 + \sqrt{5}}{2} \text{ and } \bar{\varphi} = \frac{1 - \sqrt{5}}{2}.$$

Can we compute φ^N and $\bar{\varphi}^N$ in $O(\log N)$ with fast exp, yielding an $O(\log N)$ solution...?

- φ is irrational, and processing non-integers programmatically isn't ideal.
- Especially for large N , a lot of error can accumulate...

Fibonacci – Initial Thoughts

Since $N \sim 10^{18}$, we're looking for an $O(1)$ or $O(\log N)$ solution... how???

Recall that there *is* a closed form solution to the linear recurrence:

$$F_N = \frac{\varphi^N - \bar{\varphi}^N}{\sqrt{5}}, \text{ where } \varphi = \frac{1 + \sqrt{5}}{2} \text{ and } \bar{\varphi} = \frac{1 - \sqrt{5}}{2}.$$

Can we compute φ^N and $\bar{\varphi}^N$ in $O(\log N)$ with fast exp, yielding an $O(\log N)$ solution...?

- φ is irrational, and processing non-integers programmatically isn't ideal.
- Especially for large N , a lot of error can accumulate...
- ...also, modding a float by $10^9 + 7$ is itself an issue...

Fibonacci – Initial Thoughts

Since $N \sim 10^{18}$, we're looking for an $O(1)$ or $O(\log N)$ solution... how???

Recall that there *is* a closed form solution to the linear recurrence:

$$F_N = \frac{\varphi^N - \bar{\varphi}^N}{\sqrt{5}}, \text{ where } \varphi = \frac{1 + \sqrt{5}}{2} \text{ and } \bar{\varphi} = \frac{1 - \sqrt{5}}{2}.$$

Can we compute φ^N and $\bar{\varphi}^N$ in $O(\log N)$ with fast exp, yielding an $O(\log N)$ solution...?

- φ is irrational, and processing non-integers programmatically isn't ideal.
- Especially for large N , a lot of error can accumulate...
- ...also, modding a float by $10^9 + 7$ is itself an issue...
- ...and there's a $\sqrt{5}$ in the denominator, so you need mod inverse of an irrational number!

Fibonacci – Initial Thoughts

Since $N \sim 10^{18}$, we're looking for an $O(1)$ or $O(\log N)$ solution... how???

Recall that there *is* a closed form solution to the linear recurrence:

$$F_N = \frac{\varphi^N - \bar{\varphi}^N}{\sqrt{5}}, \text{ where } \varphi = \frac{1 + \sqrt{5}}{2} \text{ and } \bar{\varphi} = \frac{1 - \sqrt{5}}{2}.$$

Can we compute φ^N and $\bar{\varphi}^N$ in $O(\log N)$ with fast exp, yielding an $O(\log N)$ solution...?

- φ is irrational, and processing non-integers programmatically isn't ideal.
- Especially for large N , a lot of error can accumulate...
- ...also, modding a float by $10^9 + 7$ is itself an issue...
- ...and there's a $\sqrt{5}$ in the denominator, so you need mod inverse of an irrational number!

So if we can't use the closed form, how can we possibly solve it in $\leq O(\log N)$?

Fibonacci – Encoding States

What characterises our 'position' in the problem?

- i.e. What do we need to know to move from F_k to F_{k+1} ?

Fibonacci – Encoding States

What characterises our 'position' in the problem?

- i.e. What do we need to know to move from F_k to F_{k+1} ?

Well, we probably need to know F_k and F_{k-1} .

Fibonacci – Encoding States

What characterises our 'position' in the problem?

- i.e. What do we need to know to move from F_k to F_{k+1} ?

Well, we probably need to know F_k and F_{k-1} .

- Let's encode our position (our state) then as (F_k, F_{k-1}) .

Fibonacci – Encoding States

What characterises our 'position' in the problem?

- i.e. What do we need to know to move from F_k to F_{k+1} ?

Well, we probably need to know F_k and F_{k-1} .

- Let's encode our position (our state) then as (F_k, F_{k-1}) .
- To transition, we move from (F_k, F_{k-1}) to $(F_k + F_{k-1}, F_k)$...

Fibonacci – Encoding States

What characterises our 'position' in the problem?

- i.e. What do we need to know to move from F_k to F_{k+1} ?

Well, we probably need to know F_k and F_{k-1} .

- Let's encode our position (our state) then as (F_k, F_{k-1}) .
- To transition, we move from (F_k, F_{k-1}) to $(F_k + F_{k-1}, F_k)$... this looks like a kind of linear transformation!

Fibonacci – Encoding States

What characterises our ‘position’ in the problem?

- i.e. What do we need to know to move from F_k to F_{k+1} ?

Well, we probably need to know F_k and F_{k-1} .

- Let's encode our position (our state) then as (F_k, F_{k-1}) .
- To transition, we move from (F_k, F_{k-1}) to $(F_k + F_{k-1}, F_k)$... this looks like a kind of linear transformation! In particular,

$$\begin{pmatrix} F_k \\ F_{k-1} \end{pmatrix} \mapsto \begin{pmatrix} F_k + F_{k-1} \\ F_k \end{pmatrix} \iff \begin{pmatrix} x \\ y \end{pmatrix} \mapsto \begin{pmatrix} x + y \\ x \end{pmatrix}$$

Fibonacci – Encoding States

What characterises our ‘position’ in the problem?

- i.e. What do we need to know to move from F_k to F_{k+1} ?

Well, we probably need to know F_k and F_{k-1} .

- Let’s encode our position (our state) then as (F_k, F_{k-1}) .
- To transition, we move from (F_k, F_{k-1}) to $(F_k + F_{k-1}, F_k)$... this looks like a kind of linear transformation! In particular,

$$\begin{pmatrix} F_k \\ F_{k-1} \end{pmatrix} \mapsto \begin{pmatrix} F_k + F_{k-1} \\ F_k \end{pmatrix} \iff \begin{pmatrix} x \\ y \end{pmatrix} \mapsto \begin{pmatrix} x + y \\ x \end{pmatrix}$$

- This will be covered in more details later, but we can represent this with the matrix

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}, \quad \text{i.e.} \quad A \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} x + y \\ x \end{pmatrix}$$

Fibonacci – Matrix Powers

So moving from the k 'th Fibonacci number to the $k + 1$ 'th just looks like left-multiplying our state vector $(F_k, F_{k-1})^\top$ by A .

Fibonacci – Matrix Powers

So moving from the k 'th Fibonacci number to the $k + 1$ 'th just looks like left-multiplying our state vector $(F_k, F_{k-1})^\top$ by A .

- We start with the state vector $(F_1, F_0) = (1, 0) = \mathbf{e}_1$.

Fibonacci – Matrix Powers

So moving from the k 'th Fibonacci number to the $k + 1$ 'th just looks like left-multiplying our state vector $(F_k, F_{k-1})^\top$ by A .

- We start with the state vector $(F_1, F_0) = (1, 0) = \mathbf{e}_1$.
- To get the 2nd Fibonacci number, we compute

$$A \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} F_1 + F_0 \\ F_1 \end{pmatrix} = \begin{pmatrix} F_2 \\ F_1 \end{pmatrix}.$$

Fibonacci – Matrix Powers

So moving from the k 'th Fibonacci number to the $k + 1$ 'th just looks like left-multiplying our state vector $(F_k, F_{k-1})^\top$ by A .

- We start with the state vector $(F_1, F_0) = (1, 0) = \mathbf{e}_1$.
- To get the 2nd Fibonacci number, we compute

$$A \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} F_1 + F_0 \\ F_1 \end{pmatrix} = \begin{pmatrix} F_2 \\ F_1 \end{pmatrix}.$$

- To get the 3rd Fibonacci number, we compute

$$A \begin{pmatrix} F_2 \\ F_1 \end{pmatrix} = A \left(A \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} \right) = A^2 \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} F_3 \\ F_2 \end{pmatrix}.$$

Fibonacci – Matrix Powers

So moving from the k 'th Fibonacci number to the $k + 1$ 'th just looks like left-multiplying our state vector $(F_k, F_{k-1})^\top$ by A .

- We start with the state vector $(F_1, F_0) = (1, 0) = \mathbf{e}_1$.
- To get the 2nd Fibonacci number, we compute

$$A \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} F_1 + F_0 \\ F_1 \end{pmatrix} = \begin{pmatrix} F_2 \\ F_1 \end{pmatrix}.$$

- To get the 3rd Fibonacci number, we compute

$$A \begin{pmatrix} F_2 \\ F_1 \end{pmatrix} = A \left(A \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} \right) = A^2 \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} F_3 \\ F_2 \end{pmatrix}.$$

- So actually, in general the N 'th Fibonacci number is just the x-coordinate of

$$A^{N-1} \begin{pmatrix} F_1 \\ F_0 \end{pmatrix}.$$

Fibonacci – Solution

We are finally ready to solve this problem!

Fibonacci – Solution

We are finally ready to solve this problem!

- The k 'th Fibonacci number is the top-left entry of A^{k-1} (for $k > 1$).

Fibonacci – Solution

We are finally ready to solve this problem!

- The k 'th Fibonacci number is the top-left entry of A^{k-1} (for $k > 1$).
- So, the problem reduces to computing A^{N-1} in $O(\log N)$.

Fibonacci – Solution

We are finally ready to solve this problem!

- The k 'th Fibonacci number is the top-left entry of A^{k-1} (for $k > 1$).
- So, the problem reduces to computing A^{N-1} in $O(\log N)$.
 - To do this, recall our **fast exponentiation** technique :)

Fibonacci – Solution

We are finally ready to solve this problem!

- The k 'th Fibonacci number is the top-left entry of A^{k-1} (for $k > 1$).
- So, the problem reduces to computing A^{N-1} in $O(\log N)$.
 - To do this, recall our **fast exponentiation** technique :)
 - Recursively, let's split A^k into $A^{\lfloor k/2 \rfloor} A^{\lfloor k/2 \rfloor}$ if k is even, and $A^{\lfloor k/2 \rfloor} A^{\lfloor k/2 \rfloor} A$ otherwise.

Fibonacci – Solution

We are finally ready to solve this problem!

- The k 'th Fibonacci number is the top-left entry of A^{k-1} (for $k > 1$).
- So, the problem reduces to computing A^{N-1} in $O(\log N)$.
 - To do this, recall our **fast exponentiation** technique :)
 - Recursively, let's split A^k into $A^{\lfloor k/2 \rfloor} A^{\lfloor k/2 \rfloor}$ if k is even, and $A^{\lfloor k/2 \rfloor} A^{\lfloor k/2 \rfloor} A$ otherwise.
 - Since A and its powers are always 2×2 matrices, multiplying them is $O(2^3) = O(8)$, which is like $O(1)$ anyway.
 - As we do $\log N$ multiplications for A^N , our final time complexity is $O(\log N)$!

Fibonacci – Solution

We are finally ready to solve this problem!

- The k 'th Fibonacci number is the top-left entry of A^{k-1} (for $k > 1$).
- So, the problem reduces to computing A^{N-1} in $O(\log N)$.
 - To do this, recall our **fast exponentiation** technique :)
 - Recursively, let's split A^k into $A^{\lfloor k/2 \rfloor} A^{\lfloor k/2 \rfloor}$ if k is even, and $A^{\lfloor k/2 \rfloor} A^{\lfloor k/2 \rfloor} A$ otherwise.
 - Since A and its powers are always 2×2 matrices, multiplying them is $O(2^3) = O(8)$, which is like $O(1)$ anyway.
 - As we do $\log N$ multiplications for A^N , our final time complexity is $O(\log N)$!

We can now find the N 'th Fibonacci number in $O(\log N)$, yay! In fact, for big values of N , the bottleneck becomes dividing N by 2. The interviewers will be very impressed I'm sure.

Fibonacci – Solution

We are finally ready to solve this problem!

- The k 'th Fibonacci number is the top-left entry of A^{k-1} (for $k > 1$).
- So, the problem reduces to computing A^{N-1} in $O(\log N)$.
 - To do this, recall our **fast exponentiation** technique :)
 - Recursively, let's split A^k into $A^{\lfloor k/2 \rfloor} A^{\lfloor k/2 \rfloor}$ if k is even, and $A^{\lfloor k/2 \rfloor} A^{\lfloor k/2 \rfloor} A$ otherwise.
 - Since A and its powers are always 2×2 matrices, multiplying them is $O(2^3) = O(8)$, which is like $O(1)$ anyway.
 - As we do $\log N$ multiplications for A^N , our final time complexity is $O(\log N)$!

We can now find the N 'th Fibonacci number in $O(\log N)$, yay! In fact, for big values of N , the bottleneck becomes dividing N by 2. The interviewers will be very impressed I'm sure.

In general, linear recurrences of order N can be represented as the exponentiation of an $N \times N$ matrix K times.

Fibonacci – Solution

We are finally ready to solve this problem!

- The k 'th Fibonacci number is the top-left entry of A^{k-1} (for $k > 1$).
- So, the problem reduces to computing A^{N-1} in $O(\log N)$.
 - To do this, recall our **fast exponentiation** technique :)
 - Recursively, let's split A^k into $A^{\lfloor k/2 \rfloor} A^{\lfloor k/2 \rfloor}$ if k is even, and $A^{\lfloor k/2 \rfloor} A^{\lfloor k/2 \rfloor} A$ otherwise.
 - Since A and its powers are always 2×2 matrices, multiplying them is $O(2^3) = O(8)$, which is like $O(1)$ anyway.
 - As we do $\log N$ multiplications for A^N , our final time complexity is $O(\log N)$!

We can now find the N 'th Fibonacci number in $O(\log N)$, yay! In fact, for big values of N , the bottleneck becomes dividing N by 2. The interviewers will be very impressed I'm sure.

In general, linear recurrences of order N can be represented as the exponentiation of an $N \times N$ matrix K times. Multiplying an $N \times N$ matrix is slow – practically around $O(N^{2.807})$.

Fibonacci – Solution

We are finally ready to solve this problem!

- The k 'th Fibonacci number is the top-left entry of A^{k-1} (for $k > 1$).
- So, the problem reduces to computing A^{N-1} in $O(\log N)$.
 - To do this, recall our **fast exponentiation** technique :)
 - Recursively, let's split A^k into $A^{\lfloor k/2 \rfloor} A^{\lfloor k/2 \rfloor}$ if k is even, and $A^{\lfloor k/2 \rfloor} A^{\lfloor k/2 \rfloor} A$ otherwise.
 - Since A and its powers are always 2×2 matrices, multiplying them is $O(2^3) = O(8)$, which is like $O(1)$ anyway.
 - As we do $\log N$ multiplications for A^N , our final time complexity is $O(\log N)$!

We can now find the N 'th Fibonacci number in $O(\log N)$, yay! In fact, for big values of N , the bottleneck becomes dividing N by 2. The interviewers will be very impressed I'm sure.

In general, linear recurrences of order N can be represented as the exponentiation of an $N \times N$ matrix K times. Multiplying an $N \times N$ matrix is slow – practically around $O(N^{2.807})$.

- We call the $O(N^{\text{blah}} \log K)$ technique for matrix powers **fast matrix exponentiation**.

Outline

- 1 Housekeeping
- 2 Gaussian Elimination
- 3 Tricks With A^N
- 4 Diagonalisation**
- 5 XOR Basis
- 6 Jordan Canonical Form and Matrix Exponential
- 7 Linear Transformations
- 8 Appendix and resources

Matrix Diagonalisation (Prior Knowledge)

Motivation

Matrix Diagonalisation is a method of quickly exponentiating matrices without brute force multiplication. It saves computation time when it comes to large matrices

Diagonalisation Definition

Definition

A square matrix, $A \in \mathbb{M}_{n,n}(\mathbb{F})$ can be 'diagonalised' if it can be decomposed into

$$A = PDP^{-1},$$

where P is an invertible matrix of distinct eigenvectors and D is the diagonal matrix which is defined as $D = (\lambda_1, \dots, \lambda_n)^T I_{n,n}$ where $\lambda_1, \dots, \lambda_n$ are eigenvalues and $I_{m,m}$ is the n th dimensional identity matrix.

Conditions for Matrix Diagonalisation

To determine if a matrix, A can be diagonalised we have the following conditions:

Conditions for Matrix Diagonalisation

To determine if a matrix, A can be diagonalised we have the following conditions:

- A is a square matrix.
- A must have $\dim(A)$ independent eigenvectors.

Conditions for Matrix Diagonalisation

To determine if a matrix, A can be diagonalised we have the following conditions:

- A is a square matrix.
- A must have $\dim(A)$ independent eigenvectors.

But what are eigenvalues and eigenvectors?

Conditions for Matrix Diagonalisation

To determine if a matrix, A can be diagonalised we have the following conditions:

- A is a square matrix.
- A must have $\dim(A)$ independent eigenvectors.

But what are eigenvalues and eigenvectors?

Definition

For matrices $A \in M_{n,n}(\mathbb{F}_1)$ with scalar field $\mathbb{F}_2$, eigenvalues are all scalars $\lambda \in \mathbb{F}_1$ and eigenvectors are all distinct vectors $\mathbf{v} \in \ker(A - \lambda I)$ that satisfy the following equation

$$A\mathbf{v} = \lambda\mathbf{v}.$$

Conditions for Matrix Diagonalisation

To determine if a matrix, A can be diagonalised we have the following conditions:

- A is a square matrix.
- A must have $\dim(A)$ independent eigenvectors.

But what are eigenvalues and eigenvectors?

Definition

For matrices $A \in M_{n,n}(\mathbb{F}_1)$ with scalar field $\mathbb{F}_2$, eigenvalues are all scalars $\lambda \in \mathbb{F}_1$ and eigenvectors are all distinct vectors $\mathbf{v} \in \ker(A - \lambda I)$ that satisfy the following equation

$$A\mathbf{v} = \lambda\mathbf{v}.$$

More generally, we look for all k vectors s.t. $(A - \lambda I)^k \mathbf{v} = \mathbf{0}$.

Conditions for Matrix Diagonalisation

To determine if a matrix, A can be diagonalised we have the following conditions:

- A is a square matrix.
- A must have $\dim(A)$ independent eigenvectors.

But what are eigenvalues and eigenvectors?

Definition

For matrices $A \in M_{n,n}(\mathbb{F}_1)$ with scalar field $\mathbb{F}_2$, eigenvalues are all scalars $\lambda \in \mathbb{F}_1$ and eigenvectors are all distinct vectors $\mathbf{v} \in \ker(A - \lambda I)$ that satisfy the following equation

$$A\mathbf{v} = \lambda\mathbf{v}.$$

More generally, we look for all k vectors s.t. $(A - \lambda I)^k \mathbf{v} = \mathbf{0}$.

Eigenvalues have the special property that $\sum_{i=1}^n \lambda_i = \text{Tr}(A)$, and $\prod_{i=1}^n \lambda_i = \det(A)$.

Procedure for Matrix Diagonalisation

To diagonalise a matrix $A \in \mathbb{M}_{n,n}(\mathbb{F}_1, \mathbb{F}_2)$, we must find its eigenvalues and eigenvectors:

Procedure for Matrix Diagonalisation

To diagonalise a matrix $A \in \mathbb{M}_{n,n}(\mathbb{F}_1, \mathbb{F}_2)$, we must find its eigenvalues and eigenvectors:

1. Finding Eigenvalues

We find all $\lambda \in \mathbb{F}_2$ that are the solution to $\det(A - \lambda I) = 0$.

This condition is derived from the definition of when eigenvalues are,

$$\begin{aligned} A\mathbf{v} &= \lambda\mathbf{v} \\ (A - \lambda I)\mathbf{v} &= \mathbf{0}, \end{aligned}$$

such that we have $\det(A - \lambda I) = 0$ as we do not want the trivial $\mathbf{v} = \mathbf{0}$ which would require $A - \lambda I$ to be invertible and have a non-zero determinant.

Procedure for Matrix Diagonalisation

2. Finding Eigenvectors

We substitute all of our eigenvalues λ_i 's for $i = 1$ to p where p is the number of eigenvalues we have found into $A\mathbf{v} = \lambda\mathbf{v}$

Then after Gaussian elimination, we simplify to find $\mathbf{v}$ whose entries are usually scalars of each other.

Procedure for Matrix Diagonalisation

2. Finding Eigenvectors

We substitute all of our eigenvalues λ_i 's for $i = 1$ to p where p is the number of eigenvalues we have found into $A\mathbf{v} = \lambda\mathbf{v}$

Then after Gaussian elimination, we simplify to find $\mathbf{v}$ whose entries are usually scalars of each other.

Procedure for Matrix Diagonalisation

3. Constructing the Diagonalised Form

If we have $|\{\mathbf{v} : \mathbf{v} \in \ker(A - \lambda I)\}| < \dim(A)$, we cannot diagonalise this matrix, otherwise we have for $A = PDP^{-1}$,

$$P = [\mathbf{v}_1 | \dots | \mathbf{v}_n], \quad D = (\lambda_1, \dots, \lambda_n)^T I_{n,n},$$

where $\lambda_1, \dots, \lambda_n$ are not necessarily be distinct but in same order as $\mathbf{v}$'s in P .

Simple Example of Matrix Diagonalisation

Example

Diagonalise $A = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$. Note that the matrix is square so we can try diagonalising.

Simple Example of Matrix Diagonalisation

Example

Diagonalise $A = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$. Note that the matrix is square so we can try diagonalising.

Simple Example of Matrix Diagonalisation

Example

Diagonalise $A = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$. Note that the matrix is square so we can try diagonalising.

Find Eigenvalues: Let λ be the eigenvalue, solve $\det(A - \lambda I) = 0$ which simplifies to $(1 - \lambda)^2 - 1 = 0$. Solving for λ , we have $\lambda = 0, 2$ which we will substitute into $A\mathbf{v} = \lambda\mathbf{v}$.

Simple Example of Matrix Diagonalisation

Example

Diagonalise $A = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$. Note that the matrix is square so we can try diagonalising.

Find Eigenvalues: Let λ be the eigenvalue, solve $\det(A - \lambda I) = 0$ which simplifies to $(1 - \lambda)^2 - 1 = 0$. Solving for λ , we have $\lambda = 0, 2$ which we will substitute into $A\mathbf{v} = \lambda\mathbf{v}$.

Finding Eigenvectors: (See whiteboard), this method gives us the eigenvectors $(1, 1)^T$ for $\lambda = 0$ and $(1, -1)^T$ for $\lambda = 2$.

Diagonalising: Hence, $A = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \cdot \begin{bmatrix} 0 & 0 \\ 0 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}^{-1}$,

Simple Example of Matrix Diagonalisation

Example

Diagonalise $A = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$. Note that the matrix is square so we can try diagonalising.

Find Eigenvalues: Let λ be the eigenvalue, solve $\det(A - \lambda I) = 0$ which simplifies to $(1 - \lambda)^2 - 1 = 0$. Solving for λ , we have $\lambda = 0, 2$ which we will substitute into $A\mathbf{v} = \lambda\mathbf{v}$.

Finding Eigenvectors: (See whiteboard), this method gives us the eigenvectors $(1, 1)^T$ for $\lambda = 0$ and $(1, -1)^T$ for $\lambda = 2$.

Diagonalising: Hence, $A = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \cdot \begin{bmatrix} 0 & 0 \\ 0 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}^{-1}$,

where $\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}^{-1} = -\frac{1}{1} \begin{bmatrix} -1 & -1 \\ -1 & 1 \end{bmatrix}$ by the $\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$ formula.

Tasty Example involving Matrix Diagonalisation

Example

SMMC 2025 A Q3 (Simplified): Consider the matrix

$$M = \begin{bmatrix} t+1 & t \\ 1 & 1 \end{bmatrix}$$

whose entries are polynomials in the variable t . Let $a(t)$ denote the polynomial in the first row and first column of M^{2025} and let $b(t)$ denote the polynomial in the first row and second column of M^{2025} .

Prove $a(t)$ has 2025 distinct real roots $\alpha_1, \alpha_2, \dots, \alpha_{2025}$ and that $b(t)$ has 2025 distinct real roots $\beta_1, \beta_2, \dots, \beta_{2025}$.

Tasty Example involving Matrix Diagonalisation

Example

The simplest way to begin this problem is to compute M^{2025} and we can achieve that with diagonalisation.

Tasty Example involving Matrix Diagonalisation

Example

The simplest way to begin this problem is to compute M^{2025} and we can achieve that with diagonalisation.

Find eigenvalues: Let $\det(M - \lambda I) = 0$, we have $(t + 1 - \lambda)(1 - \lambda) - t = 0$ which simplifies to $\lambda^2 - (t + 2)\lambda + 1 = 0$.

Tasty Example involving Matrix Diagonalisation

Example

The simplest way to begin this problem is to compute M^{2025} and we can achieve that with diagonalisation.

Find eigenvalues: Let $\det(M - \lambda I) = 0$, we have $(t + 1 - \lambda)(1 - \lambda) - t = 0$ which simplifies to $\lambda^2 - (t + 2)\lambda + 1 = 0$.

Let $t = \mu + \mu^{-1} - 2$ so that

$$\begin{aligned}\lambda^2 - (\mu + \mu^{-1})\lambda + 1 &= 0, \\ (\lambda - \mu)(\lambda - \mu^{-1}) &= 0\end{aligned}$$

Tasty Example involving Matrix Diagonalisation

Example

The simplest way to begin this problem is to compute M^{2025} and we can achieve that with diagonalisation.

Find eigenvalues: Let $\det(M - \lambda I) = 0$, we have $(t + 1 - \lambda)(1 - \lambda) - t = 0$ which simplifies to $\lambda^2 - (t + 2)\lambda + 1 = 0$.

Let $t = \mu + \mu^{-1} - 2$ so that

$$\begin{aligned}\lambda^2 - (\mu + \mu^{-1})\lambda + 1 &= 0, \\ (\lambda - \mu)(\lambda - \mu^{-1}) &= 0\end{aligned}$$

This simplification enables us neatly find the eigenvalues $\lambda = \mu, 1/\mu$.

Tasty Example involving Matrix Diagonalisation

Example

Find eigenvectors: We simply substitute our λ values into $M\mathbf{v} = \lambda\mathbf{v}$, express in $(M - \lambda I)\mathbf{v} = 0$ which gives us the following two systems to solve

$$\begin{bmatrix} t + 1 - \mu & t & | & 0 \\ 1 & 1 - \mu & | & 0 \end{bmatrix} \quad (1), \quad \begin{bmatrix} t + 1 - 1/\mu & t & | & 0 \\ 1 & 1 - 1/\mu & | & 0 \end{bmatrix} \quad (2).$$

Tasty Example involving Matrix Diagonalisation

Example

Find eigenvectors: We simply substitute our λ values into $M\mathbf{v} = \lambda\mathbf{v}$, express in $(M - \lambda I)\mathbf{v} = 0$ which gives us the following two systems to solve

$$\begin{bmatrix} t + 1 - \mu & t & | & 0 \\ 1 & 1 - \mu & | & 0 \end{bmatrix} \quad (1), \quad \begin{bmatrix} t + 1 - 1/\mu & t & | & 0 \\ 1 & 1 - 1/\mu & | & 0 \end{bmatrix} \quad (2).$$

Solving (1) we have $v_1 = (1 - \mu) v_2$ so $\mathbf{v}_1 = (1, 1 - \mu)^T$.

With (2), $v_1 = (1 - 1/\mu) v_2 = (\mu - 1)/\mu v_2$ so $\mathbf{v}_2 = (\mu, 1 - \mu)^T$.

Tasty Example involving Matrix Diagonalisation

Example

Find eigenvectors: We simply substitute our λ values into $M\mathbf{v} = \lambda\mathbf{v}$, express in $(M - \lambda I)\mathbf{v} = 0$ which gives us the following two systems to solve

$$\begin{bmatrix} t + 1 - \mu & t & | & 0 \\ 1 & 1 - \mu & | & 0 \end{bmatrix} \quad (1), \quad \begin{bmatrix} t + 1 - 1/\mu & t & | & 0 \\ 1 & 1 - 1/\mu & | & 0 \end{bmatrix} \quad (2).$$

Solving (1) we have $v_1 = (1 - \mu) v_2$ so $\mathbf{v}_1 = (1, 1 - \mu)^T$.

With (2), $v_1 = (1 - 1/\mu) v_2 = (\mu - 1)/\mu v_2$ so $\mathbf{v}_2 = (\mu, 1 - \mu)^T$.

So our $P = [\mathbf{v}_1 | \mathbf{v}_2]$, $D = (\mu, 1/\mu)^T I$.

Tasty Example involving Matrix Diagonalisation

Example

Find eigenvectors: We simply substitute our λ values into $M\mathbf{v} = \lambda\mathbf{v}$, express in $(M - \lambda I)\mathbf{v} = 0$ which gives us the following two systems to solve

$$\begin{bmatrix} t + 1 - \mu & t & | & 0 \\ 1 & 1 - \mu & | & 0 \end{bmatrix} \quad (1), \quad \begin{bmatrix} t + 1 - 1/\mu & t & | & 0 \\ 1 & 1 - 1/\mu & | & 0 \end{bmatrix} \quad (2).$$

Solving (1) we have $v_1 = (1 - \mu) v_2$ so $\mathbf{v}_1 = (1, 1 - \mu)^T$.

With (2), $v_1 = (1 - 1/\mu) v_2 = (\mu - 1)/\mu v_2$ so $\mathbf{v}_2 = (\mu, 1 - \mu)^T$.

So our $P = [\mathbf{v}_1 | \mathbf{v}_2]$, $D = (\mu, 1/\mu)^T I$.

For P^{-1} we use the formula $\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$, so $P^{-1} = \frac{1}{(\mu-1)^2} \begin{bmatrix} 1 - \mu & -\mu \\ \mu - 1 & -1 \end{bmatrix}$.

Tasty Example involving Matrix Diagonalisation

Example

Now, we compute M^{2025} and then simply manually multiply the first two iterations to get $a(t) = \frac{\mu^n(\mu-1) - (\mu-1)/\mu^n}{\mu^2-1} = \frac{\mu^{2n+1}+1}{\mu^n(\mu+1)}$ and $b(t) = \frac{(\mu-1)(\mu^{2n}-1)}{\mu^n(\mu+1)}$.

Tasty Example involving Matrix Diagonalisation

Example

Now, we compute M^{2025} and then simply manually multiply the first two iterations to get $a(t) = \frac{\mu^n(\mu-1) - (\mu-1)/\mu^n}{\mu^2-1} = \frac{\mu^{2n+1}+1}{\mu^n(\mu+1)}$ and $b(t) = \frac{(\mu-1)(\mu^{2n}-1)}{\mu^n(\mu+1)}$.

Then we solve $a(t) = 0$ and $b(t) = 0$ to find roots where we realise $\mu^n = 1$ or $\mu^{2n} = 1$ need to be solved.

Tasty Example involving Matrix Diagonalisation

Example

Now, we compute M^{2025} and then simply manually multiply the first two iterations to get $a(t) = \frac{\mu^n(\mu-1) - (\mu-1)/\mu^n}{\mu^2-1} = \frac{\mu^{2n+1}+1}{\mu^n(\mu+1)}$ and $b(t) = \frac{(\mu-1)(\mu^{2n}-1)}{\mu^n(\mu+1)}$.

Then we solve $a(t) = 0$ and $b(t) = 0$ to find roots where we realise $\mu^n = 1$ or $\mu^{2n} = 1$ need to be solved.

As this is reminiscent of Roots of Unity, let $\mu = \exp(i\theta)$ for real θ , $\mu = \exp\left(\frac{\pi ik}{2n+1}\right)$ where $k = 1, 3, \dots, 2n-1$ (unique roots of unity) for $a(t)$ and $\mu = \exp\left(\frac{\pi il}{n}\right)$ for $l = 0, \dots, n-1$ (unique roots of unity) for $b(t)$.

Tasty Example involving Matrix Diagonalisation

Example

Now, we compute M^{2025} and then simply manually multiply the first two iterations to get $a(t) = \frac{\mu^n(\mu-1) - (\mu-1)/\mu^n}{\mu^2-1} = \frac{\mu^{2n+1}-1}{\mu^n(\mu+1)}$ and $b(t) = \frac{(\mu-1)(\mu^{2n}-1)}{\mu^n(\mu+1)}$.

Then we solve $a(t) = 0$ and $b(t) = 0$ to find roots where we realise $\mu^n = 1$ or $\mu^{2n} = 1$ need to be solved.

As this is reminiscent of Roots of Unity, let $\mu = \exp(i\theta)$ for real θ , $\mu = \exp\left(\frac{\pi ik}{2n+1}\right)$ where $k = 1, 3, \dots, 2n-1$ (unique roots of unity) for $a(t)$ and $\mu = \exp\left(\frac{\pi il}{n}\right)$ for $l = 0, \dots, n-1$ (unique roots of unity) for $b(t)$.

Now we substitute these values into our substitution for t to get $t = 2 \cos\left(\frac{\pi k}{2n+1}\right) - 2$ and for $b(t)$, $t = 2 \cos\left(\frac{\pi l}{n}\right) - 2$.

Outline

- 1 Housekeeping
- 2 Gaussian Elimination
- 3 Tricks With A^N
- 4 Diagonalisation
- 5 XOR Basis**
- 6 Jordan Canonical Form and Matrix Exponential
- 7 Linear Transformations
- 8 Appendix and resources

Revision

To understand **XOR Basis**, we first need to understand **basis**, which means we need to understand **linear independence** and **span**, which means we need to understand **vector spaces**, which means we need to ...

Linear Independence

A sequence of vectors $v_1, v_2, \dots, v_n$, is **linearly independent** if and only if the only combination that gives the zero vector is $0v_1 + 0v_2 + \dots + 0v_n$.

Revision

To understand **XOR Basis**, we first need to understand **basis**, which means we need to understand **linear independence** and **span**, which means we need to understand **vector spaces**, which means we need to ...

Linear Independence

A sequence of vectors $v_1, v_2, \dots, v_n$, is **linearly independent** if and only if the only combination that gives the zero vector is $0v_1 + 0v_2 + \dots + 0v_n$.

Uniqueness of Linear Combinations

Let S be a finite, non-empty set of vectors in a vector space and let v be a vector which can be written as a linear combination of S . Then the values of the scalars in the linear combination for v are unique if and only if S is a linearly independent set.

Revision

Span

Let $S = v_1, \dots, v_n$ be a finite set of vectors in a vector space V over a field $\mathbb{F}$. Then the span of the set S is the set of all linear combinations of S .

Revision

Span

Let $S = v_1, \dots, v_n$ be a finite set of vectors in a vector space V over a field $\mathbb{F}$. Then the span of the set S is the set of all linear combinations of S .

Basis

A set of vectors B in a vector space V is called a basis for V if:

- 1 B is a linearly independent set, and
- 2 B is a spanning set for V (that is, $\text{span}(B) = V$)

XOR Closure

Problem

You are given a set S of N distinct integers, where $1 \leq N \leq 10^5$.

XOR Closure

Problem

You are given a set S of N distinct integers, where $1 \leq N \leq 10^5$.

Call the set *closed* if

$$A, B \in S \implies A \oplus B \in S.$$

XOR Closure

Problem

You are given a set S of N distinct integers, where $1 \leq N \leq 10^5$.

Call the set *closed* if

$$A, B \in S \implies A \oplus B \in S.$$

What is the minimum number of integers we must add to S to make it closed?

Building the Basis – One Element at a Time

It suffices to insert the integers of X one by one.

Building the Basis – One Element at a Time

It suffices to insert the integers of X one by one.

- If $x_{i+1} \in \text{span}(x_1, \dots, x_i)$, adding it leaves the span unchanged.

Building the Basis – One Element at a Time

It suffices to insert the integers of X one by one.

- If $x_{i+1} \in \text{span}(x_1, \dots, x_i)$, adding it leaves the span unchanged.
- Otherwise the basis grows by one, and the span doubles in size.

Building the Basis – One Element at a Time

It suffices to insert the integers of X one by one.

- If $x_{i+1} \in \text{span}(x_1, \dots, x_i)$, adding it leaves the span unchanged.
- Otherwise the basis grows by one, and the span doubles in size.

Question

How can we quickly test whether $x_{i+1} \in \text{span}(x_1, \dots, x_i)$? Checking all 2^i linear combinations is far too slow.

Most Significant Bit

For a positive integer x , let $\text{msb}(x)$ be the index of its most significant bit (one less than its length in base 2).

Most Significant Bit

For a positive integer x , let $\text{msb}(x)$ be the index of its most significant bit (one less than its length in base 2).

Example

$$\text{msb}(4) = \text{msb}(7) = 2, \quad \text{msb}(3) = 1, \quad \text{msb}(1) = 0.$$

Most Significant Bit

For a positive integer x , let $\text{msb}(x)$ be the index of its most significant bit (one less than its length in base 2).

Example

$$\text{msb}(4) = \text{msb}(7) = 2, \quad \text{msb}(3) = 1, \quad \text{msb}(1) = 0.$$

Easier version

Suppose we keep a basis $\{v_1, \dots, v_n\}$ with

$$\text{msb}(v_1) > \text{msb}(v_2) > \dots > \text{msb}(v_n).$$

Can we easily test whether a vector a is in $\text{span}(\{v_1, \dots, v_n\})$?

Is $20 \in \text{span}(\{26, 15, 3, 1\})$?

$$26 = 11010_2$$

$$15 = 01111_2$$

$$3 = 00011_2$$

$$1 = 00001_2$$

$$20 = 10100_2$$

Is $20 \in \text{span}(\{26, 15, 3, 1\})$?

$$26 = 11010_2$$

$$15 = 01111_2$$

$$3 = 00011_2$$

$$1 = 00001_2$$

$$20 = 10100_2$$

Yes: $20 = 26 \oplus 15 \oplus 1$.

The Membership Test

Solution

Let $A = a$. Iterate over i from $1 \dots n$. If $A \oplus v_i < A$, replace A with $A \oplus v_i$. If $A = 0$ at the end, then $a \in \text{span}(\{v_1, \dots, v_n\})$.

The Membership Test

Solution

Let $A = a$. Iterate over i from $1 \dots n$. If $A \oplus v_i < A$, replace A with $A \oplus v_i$. If $A = 0$ at the end, then $a \in \text{span}(\{v_1, \dots, v_n\})$.

More generally

This computes the *minimum* value of $a \oplus v$ over all $v \in \text{span}(\{v_1, \dots, v_n\})$.

Why Is This Correct?

A value is added *if and only if* it is independent — these two directions are all we need.

Reduction only ever XORs basis elements into A , so at the end $A = a \oplus v$ for some $v \in \text{span}$.

Why Is This Correct?

A value is added *if and only if* it is independent — these two directions are all we need.

Reduction only ever XORs basis elements into A , so at the end $A = a \oplus v$ for some $v \in \text{span}$.

Independent $\Rightarrow$ added

Reaching $A = 0$ would mean $a = v \in \text{span}$. So if $a \notin \text{span}$, then A never becomes 0: it survives and is added.

Why Is This Correct?

A value is added *if and only if* it is independent — these two directions are all we need.
Reduction only ever XORs basis elements into A , so at the end $A = a \oplus v$ for some $v \in \text{span}$.

Independent $\Rightarrow$ added

Reaching $A = 0$ would mean $a = v \in \text{span}$. So if $a \notin \text{span}$, then A never becomes 0: it survives and is added.

Added $\Rightarrow$ independent

A survivor $A \neq 0$ has had every basis msb cleared, so its top bit sits in a column no basis element leads with. Any nonzero XOR of basis elements has its top bit at some basis msb — a column A has blanked — so nothing in span equals A . Hence $a \notin \text{span}$.

Is $9 \in \text{span}(\{26, 15, 3, 1\})$?

$$26 = 11010_2$$

$$15 = 01111_2$$

$$3 = 00011_2$$

$$1 = 00001_2$$

$$9 = 01001_2$$

Is $9 \in \text{span}(\{26, 15, 3, 1\})$?

$$26 = 11010_2$$

$$15 = 01111_2$$

$$3 = 00011_2$$

$$1 = 00001_2$$

$$9 = 01001_2$$

No! But the minimum value of $9 \oplus v$ over $v \in \text{span}(\{26, 15, 3, 1\})$ is

$$9 \oplus 15 \oplus 3 \oplus 1 = 4 = 100_2.$$

Inserting 4 Into the Basis

If we add 4 and keep the sorted order:

$$26 = 11010_2$$

$$15 = 01111_2$$

$$4 = 00100_2$$

$$3 = 00011_2$$

$$1 = 00001_2$$

Inserting 4 Into the Basis

If we add 4 and keep the sorted order:

$$26 = 11010_2$$

$$15 = 01111_2$$

$$4 = 00100_2$$

$$3 = 00011_2$$

$$1 = 00001_2$$

The most significant bits are still in decreasing order!

C++ code

```
using ll = long long;           // values in  $[0, 2^{60})$ 
```

```
vector<ll> basis;
```

```
// Insert x, keeping the basis independent.
```

```
void add(ll x) {  
    for (ll b : basis) x = min(x, x ^ b);  
    if (x != 0) basis.push_back(x);  
}
```

C++ code

```
using ll = long long;           // values in  $[0, 2^{60})$ 

vector<ll> basis;

// Insert x, keeping the basis independent.
void add(ll x) {
    for (ll b : basis) x = min(x, x ^ b);
    if (x != 0) basis.push_back(x);
}
```

Call add on each value to build the basis.

C++ code

```
using ll = long long;           // values in  $[0, 2^{60})$ 

vector<ll> basis;

// Insert x, keeping the basis independent.
void add(ll x) {
    for (ll b : basis) x = min(x, x ^ b);
    if (x != 0) basis.push_back(x);
}
```

Call `add` on each value to build the basis. We don't bother keeping the basis msb-sorted — it still works!

Outline

- 1 Housekeeping
- 2 Gaussian Elimination
- 3 Tricks With A^N
- 4 Diagonalisation
- 5 XOR Basis
- 6 Jordan Canonical Form and Matrix Exponential**
- 7 Linear Transformations
- 8 Appendix and resources

Jordan Canonical Form

But what if we cannot diagonalise a matrix?

Jordan Canonical Form

But what if we cannot diagonalise a matrix? Enter the Jordan Canonical Form.

Jordan Canonical Form

But what if we cannot diagonalise a matrix? Enter the Jordan Canonical Form. But first, some background:

Definition

Let A, B be matrices, then we write

$$A \oplus B = \begin{pmatrix} A & \mathbf{0} \\ \mathbf{0} & B \end{pmatrix}$$

for the direct sum of A and B .

Generalised Eigenspaces

Definition

Let $A \in M_n(\mathbb{C})$ and λ an eigenvalue of A . Then, the generalised eigenspace of λ , written G_λ , is the set

$$G_\lambda = \{\mathbf{v} \in \mathbb{C}^n : (A - \lambda I)^k \mathbf{v} = \mathbf{0} \text{ for some } k \in \mathbb{Z}_{>0}\},$$

and the vectors in this set are called generalised eigenvectors (for λ).

Generalised Eigenspaces

Definition

We say a sequence of vectors $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k$ form a Jordan chain of length k if

$$A\mathbf{v}_1 = \lambda\mathbf{v}_1, A\mathbf{v}_i = \lambda\mathbf{v}_i + \mathbf{v}_{i-1}, i > 1,$$

where $\mathbf{v}_1$ is a λ -eigenvector of $A \in M_n(\mathbb{C})$.

Jordan Blocks

Definition

We say that the matrix $J_k(\lambda) \in M_k(\mathbb{C})$ is a Jordan block for eigenvalue λ of size k if

$$J_k(\lambda) = \begin{pmatrix} \lambda & 1 & 0 & \cdots & 0 & 0 \\ 0 & \lambda & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & \lambda & 1 \\ 0 & 0 & 0 & \cdots & 0 & \lambda \end{pmatrix}.$$

Jordan Canonical Form

Theorem

There exists a basis of generalised eigenvectors such that any matrix $A \in M_n(\mathbb{C})$ can be written as the direct sum of Jordan blocks.

Jordan Canonical Form

Theorem

There exists a basis of generalised eigenvectors such that any matrix $A \in M_n(\mathbb{C})$ can be written as the direct sum of Jordan blocks.

Example: Jordan Form and Matrix Powers

Jordan form makes it easy to compute powers of matrices.

Example

Take $A = \begin{pmatrix} 2 & 5 & -10 \\ 0 & 6 & -8 \\ 0 & 2 & -2 \end{pmatrix}$. Then $A = PJP^{-1}$, where

$$P = \begin{pmatrix} 5 & 0 & 1 \\ 4 & 1 & 1 \\ 2 & 0 & 0 \end{pmatrix}, \quad J = J_2(2) \oplus J_1(2) = \begin{pmatrix} 2 & 1 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}.$$

Hence, $A^n = PJ^nP^{-1}$, where $J^n = \begin{pmatrix} 2^n & n2^{n-1} & 0 \\ 0 & 2^n & 0 \\ 0 & 0 & 2^n \end{pmatrix}$.

Matrix Exponentiation

Motivation

The main use of matrix exponentiation is to solve multivariate systems of differential equations, however there are some other uses in probability and quantum mechanics!

Matrix Exponentiation

Motivation

The main use of matrix exponentiation is to solve multivariate systems of differential equations, however there are some other uses in probability and quantum mechanics!

Matrix Exponentiation

Example

Suppose we wish to solve the equation

$$\begin{aligned}x_1'(t) &= x_1(t) + 2x_2(t), \\x_2'(t) &= 6x_1(t) - 17x_2(t),\end{aligned}$$

Matrix Exponentiation

Example

Suppose we wish to solve the equation

$$\begin{aligned}x_1'(t) &= x_1(t) + 2x_2(t), \\x_2'(t) &= 6x_1(t) - 17x_2(t),\end{aligned}$$

then we can rewrite the equation as

$$\frac{d\mathbf{x}}{dt} = A\mathbf{x},$$

where

$$\frac{d\mathbf{x}}{dt} = \begin{pmatrix} x_1' \\ x_2' \end{pmatrix}, \quad A = \begin{pmatrix} 1 & 2 \\ 6 & -17 \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}.$$

Matrix Exponentiation Intuition

Theorem

Suppose we have a system of differential equations $\frac{d\mathbf{x}}{dt} = A\mathbf{x}$, subject to some initial conditions $\mathbf{x}(0) = \mathbf{x}_0$, then the solution to the equation is

$$\mathbf{x} = e^{At}\mathbf{x}_0.$$

Matrix Exponentiation Intuition

Theorem

Suppose we have a system of differential equations $\frac{d\mathbf{x}}{dt} = A\mathbf{x}$, subject to some initial conditions $\mathbf{x}(0) = \mathbf{x}_0$, then the solution to the equation is

$$\mathbf{x} = e^{At}\mathbf{x}_0.$$

But what even is e to the power of a matrix?

Matrix Exponentiation Definition

Definition

If A is a matrix, then we can define e^A using the Taylor expansion of the exponential function, that is

$$e^A = \sum_{n=0}^{\infty} \frac{A^n}{n!},$$

and it is easy to prove that this always converges using matrix norms.

We can get a decent approximation if we take partial sums. But, what if we want the exact value?

Computing Matrix Exponentiation

If our matrix A is diagonalisable, then computing the exponential is easy!

Theorem

Suppose A is a diagonalisable matrix with $A = PDP^{-1}$. Then

$$e^{At} = Pe^{Dt}P^{-1},$$

where

$$e^{Dt} = \begin{pmatrix} e^{\lambda_1 t} & 0 & \dots & 0 \\ 0 & e^{\lambda_2 t} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & e^{\lambda_n t} \end{pmatrix}.$$

Cayley-Hamilton Theorem

But what if the other methods are too boring? What's a more novel way of computing matrix exponentials?

Cayley-Hamilton Theorem

But what if the other methods are too boring? What's a more novel way of computing matrix exponentials? Enter the Cayley-Hamilton Theorem.

Theorem

Let $A \in M_n(\mathbb{C})$. If $p(\lambda) = \det(A - \lambda I)$, then A satisfies $p(A) = 0$.

Cayley-Hamilton Theorem

But what if the other methods are too boring? What's a more novel way of computing matrix exponentials? Enter the Cayley-Hamilton Theorem.

Theorem

Let $A \in M_n(\mathbb{C})$. If $p(\lambda) = \det(A - \lambda I)$, then A satisfies $p(A) = 0$.

Since p is always degree n , this gives us a way to rewrite A^k in terms of $1, A, A^2, \dots, A^{n-1}$ for all $k \geq n$.

Cayley-Hamilton Theorem

So we can reduce the expression to

$$e^{At} = \sum_{k=0}^{n-1} \alpha_k A^k,$$

where each eigenvalue λ_i satisfies

$$e^{\lambda_i t} = \sum_{k=0}^{n-1} \alpha_k \lambda_i^k,$$

which we solve for the coefficients (which are functions of t) to compute e^{At} .

Outline

- 1 Housekeeping
- 2 Gaussian Elimination
- 3 Tricks With A^N
- 4 Diagonalisation
- 5 XOR Basis
- 6 Jordan Canonical Form and Matrix Exponential
- 7 Linear Transformations**
- 8 Appendix and resources

Linear Transformation

Definition of Linear Transformations

Let V and W be two vector spaces over the same field $\mathbb{F}$. A function $T : V \rightarrow W$ is called a linear map or linear transformation if the following two conditions are satisfied.

- **Addition Condition.** $T(\mathbf{v} + \mathbf{u}) = T(\mathbf{v}) + T(\mathbf{u})$ for all $\mathbf{v}, \mathbf{u} \in V$, and
- **Scalar Multiplication Condition.** $T(\lambda \mathbf{v}) = \lambda T(\mathbf{v})$ for all $\lambda \in \mathbb{F}$ and $\mathbf{v} \in V$.

Linear Transformations are cool and awesome because a lot of things are linear transformations. They also preserve important vector space structures like rotation, scaling, etc etc.

Linear Transformations as Matrices

It turns out we can represent any Linear Transformation as a Matrix

General Matrix Representation Theorem

Let $T : V \rightarrow W$ be a linear map from an n -dimensional vector space V to an m -dimensional vector space W , and let $B_V = \{v_1, \dots, v_n\}$ be an ordered basis for V and $B_W = \{w_1, \dots, w_m\}$ be an ordered basis for W . Then, there is a unique $m \times n$ matrix A such that

$$[T(\mathbf{v})]_{B_W} = A[\mathbf{v}]_{B_V}.$$

It turns out the mapping of Linear Transformations to Matrices is itself a Linear Transformation. waow...

Manhattan Distance Trick

Problem

Consider N points in $\mathbb{R}^2$. What is the maximum possible Manhattan distance between two distinct points?

Recall the *Manhattan distance* between two points (x_i, y_i) and (x_j, y_j) is defined by $|x_i - x_j| + |y_i - y_j|$.

Manhattan Distance Trick

Problem

Consider N points in $\mathbb{R}^2$. What is the maximum possible Manhattan distance between two distinct points?

Recall the *Manhattan distance* between two points (x_i, y_i) and (x_j, y_j) is defined by $|x_i - x_j| + |y_i - y_j|$.

Clearly brute-forcing would take us $O(N^2)$. Could we find a more efficient algorithm?

Chebyshev Distance

Consider a different distance metric. Lets suppose we wanted to find the maximum Chebyshev distance between a set of points instead.

The Chebyshev distance is defined as:

$$d_{Chebyshev}((x_1, y_1), (x_2, y_2)) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

Chebyshev Distance

Consider a different distance metric. Lets suppose we wanted to find the maximum Chebyshev distance between a set of points instead.

The Chebyshev distance is defined as:

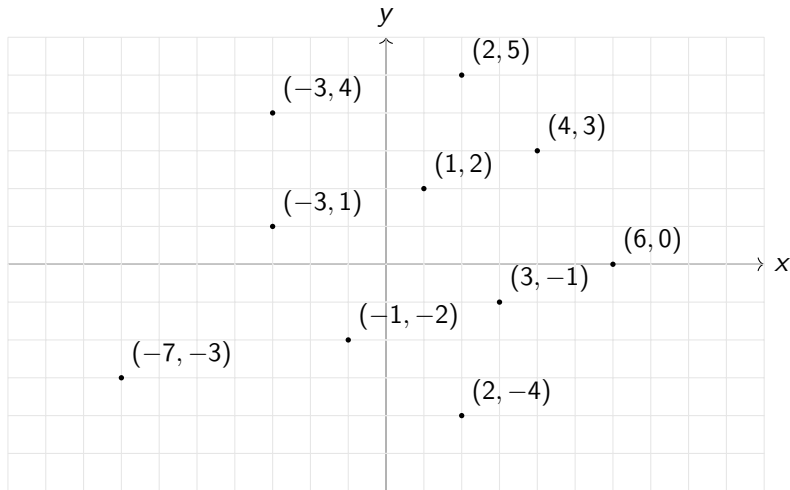
$$d_{Chebyshev}((x_1, y_1), (x_2, y_2)) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

Which makes the overall highest pairwise Chebyshev distance within a group of points as follows.

$$\max_{1 \leq i, j \leq N} \max(|x_i - x_j|, |y_i - y_j|)$$

This can be solved in $O(N)$ rather than $O(N^2)$ like for Manhattan distance. Why is this the case?

Chebyshev Distance



Manhattan Distance to Chebyshev Distance

Now is there anyway we can turn the problem of finding the maximum Manhattan distance into that of finding the maximum Chebyshev distance?

Specifically we would like to transform each point in the original problem such that:

$$d_{Chebyshev}(T(p_1), T(p_2)) = d_{Manhattan}(p_1, p_2)$$

Manhattan Distance to Chebyshev Distance

Now is there anyway we can turn the problem of finding the maximum Manhattan distance into that of finding the maximum Chebyshev distance?

Specifically we would like to transform each point in the original problem such that:

$$d_{Chebyshev}(T(p_1), T(p_2)) = d_{Manhattan}(p_1, p_2)$$

Manhattan Distance to Chebyshev Distance

Now is there anyway we can turn the problem of finding the maximum Manhattan distance into that of finding the maximum Chebyshev distance?

Specifically we would like to transform each point in the original problem such that:

$$d_{Chebyshev}(T(p_1), T(p_2)) = d_{Manhattan}(p_1, p_2)$$

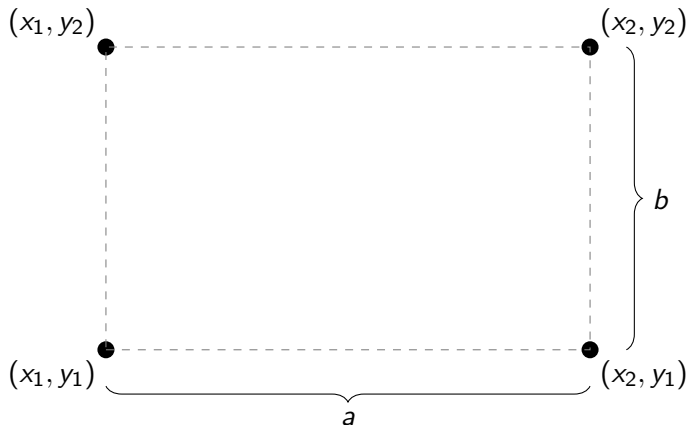
The key identity is

$$|a| + |b| = \max\{|a + b|, |a - b|\}.$$

Note if we let substitute $a = x_i - x_j$, $b = y_i - y_j$, we obtain that

$$|x_i - x_j| + |y_i - y_j| = \max\{|(x_i - x_j) + (y_i - y_j)|, |(x_i - x_j) - (y_i - y_j)|\}.$$

Visualisation of Substitution



$$|x_i - x_j| + |y_i - y_j| = \max\{|(x_i - x_j) + (y_i - y_j)|, |(x_i - x_j) - (y_i - y_j)|\}.$$

Manhattan Distance Trick

Rearranging we obtain

$$|x_i - x_j| + |y_i - y_j| = \max\{|(x_i + y_i) - (x_j + y_j)|, |(x_i - y_i) - (x_j - y_j)|\},$$

where we note each bracketed expression is intrinsic to the i th point.

So if we define $u_i = x_i + y_i$, $v_i = x_i - y_i$, then

$$\max_{1 \leq i < j \leq N} |x_i - x_j| + |y_i - y_j| = \max_{1 \leq i < j \leq N} \max\{|u_i - u_j|, |v_i - v_j|\}.$$

Note each u_i and v_i are compared only within each other. Thus the final answer is either $\max_{1 \leq i \leq N} u_i - \min_{1 \leq i \leq N} u_i$ or the analogue with v_i instead; this time complexity is $O(N)$.

Manhattan Distance Trick

Remark

What would be the motivation for the key identity?

Consider

$$p = \begin{pmatrix} x_i \\ y_i \end{pmatrix}, q = \begin{pmatrix} x_j \\ y_j \end{pmatrix},$$

and the linear transformation

$$T = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \sqrt{2} \begin{pmatrix} \cos(\frac{\pi}{4}) & -\sin(\frac{\pi}{4}) \\ \sin(\frac{\pi}{4}) & \cos(\frac{\pi}{4}) \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix},$$

that is reflection in the x -axis followed by anti-clockwise rotation by 45° .

The identity above actually corresponds to

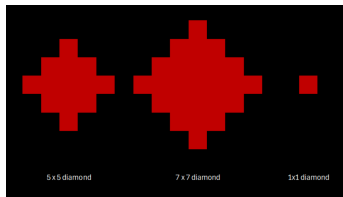
$$\|p - q\|_1 = \|T(p - q)\|_\infty$$

Diamonds!

Problem

You are given an $N \times N$ grid. Each grid cell is either black or red. What are the width of the largest red diamond in the grid?

Constraints: $N \leq 500$



Link: <https://aucpl.com/problem/holeymoley>

Diamonds!

How might we do this?

Try every centre? And spread out? This would take $O(N^2 * N^2)$ time.

Would the problem be easier if instead we were looking for the largest red square? How do we turn the diamonds into squares?

Diamonds!

How might we do this?

Try every centre? And spread out? This would take $O(N^2 * N^2)$ time.

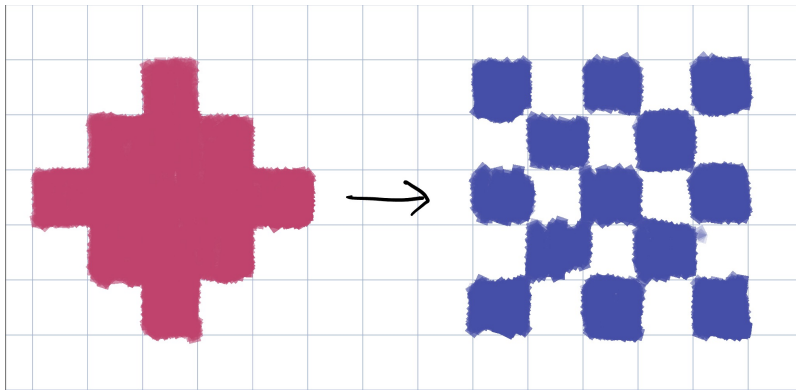
Would the problem be easier if instead we were looking for the largest red square? How do we turn the diamonds into squares?

It's the same transformation as before!

$$p = \begin{pmatrix} x_i \\ y_i \end{pmatrix}, q = \begin{pmatrix} x_j \\ y_j \end{pmatrix},$$

$$T = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \sqrt{2} \begin{pmatrix} \cos(\frac{\pi}{4}) & -\sin(\frac{\pi}{4}) \\ \sin(\frac{\pi}{4}) & \cos(\frac{\pi}{4}) \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix},$$

The Transformation

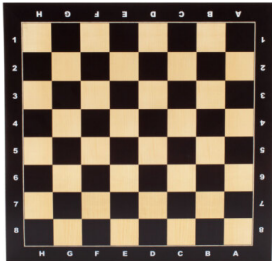


Notice that the side-length of the transformed points is equal to the diagonal of the original points (which is the value we are interested in). Why?

Now we can just look for the largest square in the transformed points.

Parity of Points after Transformation

But we need to be careful that only half of the cells in this new space will be mapped onto.



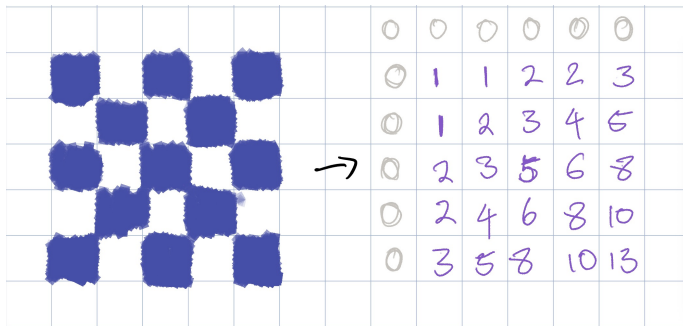
This is because $x + y$ and $x - y$ always has the same parity. So the transformed points will only land on the "white" grids on a chessboard. Essentially we can ignore the black ones.

What's the size of this chessboard?

2D Prefix Sum

A diamond of size k , has $\frac{k^2+1}{2}$ cells. Now we just need to find a square region of length k that contains $\frac{k^2+1}{2}$ red cells (note that we only care about odd values of k because diamonds always have an odd length).

We can do this via a 2D prefix sum.



Time Complexity?

- **Grid size:** The new grid has the dimensions of the original one, but rotated and scaled by a factor of $\sqrt{2}$ in each direction. This is about twice the area of the original but we need to pad it for the prefix sum. So about $4N^2$, still $O(N^2)$.
- **Prefix sum:** $O(1)$ per cell in the grid.
- **Looking for squares:** $O(N^2)$ per size of square. We can binary search.

Overall this is $O(N^2 \log(n))$.

Note the main optimisation came from the fact that having looking for squares allows us to use prefix sum which can be queried in $O(1)$.

Outline

- 1 Housekeeping
- 2 Gaussian Elimination
- 3 Tricks With A^N
- 4 Diagonalisation
- 5 XOR Basis
- 6 Jordan Canonical Form and Matrix Exponential
- 7 Linear Transformations
- 8 Appendix and resources**

Feedback!

Please fill out feedback form so we know how to run future workshops :D



(a) Feedback form



(b) Attendance form

Photo time :)

Back to Jordan Blocks

We have already seen that every matrix $A \in M_n(\mathbb{C})$ can be written as a direct sum of Jordan blocks.

Okay, so we know every matrix can be written in this form. But how do we actually find it?

Finding Jordan Blocks and Bases

Algorithm

Suppose $A \in M_n(\mathbb{C})$, then:

- 1 Compute $p(t) = \det(A - tI)$, and factorise as $p(t) = (t - \lambda_1)^{m_1} \cdots (t - \lambda_r)^{m_r}$.
- 2 For each λ_i , compute $\ker((A - \lambda_i I)^k)$ until $\ker((A - \lambda_i I)^k) = \ker((A - \lambda_i I)^{k+1})$.
- 3 For each λ_i , let $d_l = \dim(\ker((A - \lambda_i I)^l))$. Then the number of Jordan blocks for λ_i of size at least l is $d_l - d_{l-1}$, and d_1 is the total number of Jordan blocks.
- 4 To find G_{λ_i} , find vectors in $\mathbb{C}^n$ not in any of the kernels from step 2 and apply $(A - \lambda_i I)$ repeatedly to form Jordan chains, which are length at most m_i .
- 5 Repeat step 4 for each eigenvalue, which gives the bases for each G_{λ_i} .
- 6 The length of the Jordan chains correspond to the sizes of the Jordan blocks, so the Jordan form can be constructed as on the next slide.

Constructing Jordan Forms

Method

Suppose $\mathbf{v}_k \rightarrow \mathbf{v}_{k-1} \rightarrow \cdots \rightarrow \mathbf{v}_1$ is a Jordan chain for λ , then

$$J = J_k(\lambda), P = (\mathbf{v}_1 | \mathbf{v}_2 | \cdots | \mathbf{v}_k).$$

For every other Jordan chain, we direct sum the respective Jordan block to J , and adjoin the vectors in reverse order to P until $\dim J = n$. Then we can write $A = PJP^{-1}$.

Example on Jordan Forms (Expanded)

Example

Suppose we wish to find the Jordan form of

$$A = \begin{pmatrix} 2 & 5 & -10 \\ 0 & 6 & -8 \\ 0 & 2 & -2 \end{pmatrix}.$$

Then $p(t) = (t - 2)^3$ and

$$d_1 = \ker(A - 2I) = \operatorname{span} \left\{ \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 2 \\ 1 \end{pmatrix} \right\},$$

so there are at least 2 Jordan blocks of size at least 1, meaning there must be a Jordan block of size 2 and one of size 1.

Example on Jordan Forms (Expanded)

Example

Now, $(A - 2I)^2 = (A - 2I)^3 = 0$, so we pick a vector not in $\ker(A - 2I)$, such as $(0 \ 1 \ 0)^T$, and apply $(A - 2I)$, which gives us $(5 \ 4 \ 2)^T$ which lies in $\ker(A - 2I)$, so applying it one more time takes us to the zero vector. So, these 2 vectors form part of our eigenbasis, and we can pick any vector for the last one for it to be a basis, say $(1 \ 1 \ 0)^T$. Then we have that

$$P = \begin{pmatrix} 5 & 0 & 1 \\ 4 & 1 & 1 \\ 2 & 0 & 0 \end{pmatrix}, J = J_2(2) \oplus J_1(2) = \begin{pmatrix} 2 & 1 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}.$$

Uses of Jordan Forms

Don't worry if you don't understand Jordan forms, as there is a lot of theory you need to grasp them. Just know that you can get something close enough to diagonalisation for any complex matrix. In doing so, we can calculate matrix powers much easier.

Uses of Jordan Forms

Don't worry if you don't understand Jordan forms, as there is a lot of theory you need to grasp them. Just know that you can get something close enough to diagonalisation for any complex matrix. In doing so, we can calculate matrix powers much easier.

Lemma

For matrices A, B , we have that

$$(A \oplus B)^n = A^n \oplus B^n.$$

Uses of Jordan Forms

Corollary

For matrices $A_1, \dots, A_k$, we have that

$$(A_1 \oplus A_2 \oplus \dots \oplus A_k)^n = A_1^n \oplus A_2^n \oplus \dots \oplus A_k^n.$$

Uses of Jordan Forms

Corollary

For matrices $A_1, \dots, A_k$, we have that

$$(A_1 \oplus A_2 \oplus \dots \oplus A_k)^n = A_1^n \oplus A_2^n \oplus \dots \oplus A_k^n.$$

Lemma

If $J_k(\lambda)$ is a Jordan block, then $J_k(\lambda)^n$ is a matrix with $(i, j)^{th}$ entry

$$\binom{n}{j-i} \lambda^{n-j+i},$$

for $1 \leq i \leq j \leq k$.

Uses of Jordan Forms

Corollary

For matrices $A_1, \dots, A_k$, we have that

$$(A_1 \oplus A_2 \oplus \dots \oplus A_k)^n = A_1^n \oplus A_2^n \oplus \dots \oplus A_k^n.$$

Lemma

If $J_k(\lambda)$ is a Jordan block, then $J_k(\lambda)^n$ is a matrix with $(i, j)^{th}$ entry

$$\binom{n}{j-i} \lambda^{n-j+i},$$

for $1 \leq i \leq j \leq k$.

Column Method

But what if the matrix we got is not diagonalisable? How do we find the matrix exponential?

Column Method

But what if the matrix we got is not diagonalisable? How do we find the matrix exponential? Enter the column method.

Lemma

Let λ be an eigenvalue of $A \in M_n(\mathbb{C})$. Then if $\mathbf{v}$ is in the generalised eigenspace of λ , we have that

$$e^{At}\mathbf{v} = e^{\lambda t} \sum_{k=0}^{\infty} \frac{1}{k!} t^k (A - \lambda I)^k \mathbf{v},$$

and there exists some p such that $(A - \lambda I)^k \mathbf{v} = \mathbf{0}$ for all $k \geq p$.

Column Method

But what if the matrix we got is not diagonalisable? How do we find the matrix exponential? Enter the column method.

Lemma

Let λ be an eigenvalue of $A \in M_n(\mathbb{C})$. Then if $\mathbf{v}$ is in the generalised eigenspace of λ , we have that

$$e^{At}\mathbf{v} = e^{\lambda t} \sum_{k=0}^{\infty} \frac{1}{k!} t^k (A - \lambda I)^k \mathbf{v},$$

and there exists some p such that $(A - \lambda I)^k \mathbf{v} = \mathbf{0}$ for all $k \geq p$.

Using this, we can choose $\mathbf{v} = \mathbf{e}_j$ for $1 \leq j \leq n$, which we can rewrite in terms of our generalised eigenbasis to find the columns of e^{At} .

Putzer's Algorithm

But what if I can't be bothered finding the eigenbasis?

Putzer's Algorithm

But what if I can't be bothered finding the eigenbasis? Enter Putzer's Algorithm.

Putzer's Algorithm

Let $A \in M_{n,n}(\mathbb{C})$ and $\lambda_1, \dots, \lambda_n$ be its n (not necessarily distinct) eigenvalues. Then

$$e^{At} = \sum_{k=1}^n f_k(t) P_{k-1},$$

where f_k is a scalar function and P_k is a matrix, which satisfy

$$P_k = P_{k-1}(A - \lambda_k I), P_0 = I,$$

$$f_1' = \lambda_1 f_1, f_1(0) = 1,$$

$$f_k' = \lambda_k f_k + f_{k-1}, f_k(0) = 1.$$

